CÂU HỎI ÔN TẬP – LEXER

Lưu ý: các câu hỏi dưới đây chỉ có mục đích giúp sinh viên ôn tập lại chương Lexer.

Các hình ảnh có trong đề giúp sinh viên đỡ phải học thuộc khi làm đề ôn tập này. Sinh viên vẫn phải nắm vững để đi thi.

Nếu có câu hỏi nào vô tình trùng với đề thi giữa kỳ sắp tới, tác giả xin phép không chịu trách nhiệm.

Có tất cả 20 câu hỏi trắc nghiệm gồm 4 phương án trả lời A, B, C, D. Sinh viên chọn TẤT CẢ đáp án mà sinh viên cho là đúng

Câu 1: Các luật lexer trong ANTLR đôi khi sẽ có khối lệnh đằng sau, gọi là action block, được viết bằng ngôn ngữ đích (target language). Khối lệnh này có công dụng gì?

A. Thực thi đoạn lệnh trong action block khi token tương ứng được match

B. Giúp lexer thông báo lỗi cho người dùng nếu trong action block là các đoạn lệnh ném lỗi

C. Không có công dụng gì

D. Giúp người dùng có thể tùy chỉnh, xử lý linh hoạt các token được match

Đoạn code lexer sau dùng để biểu diễn số nguyên và số thực trong ngôn ngữ lập trình X

```
INT_LIT: (DEC | '0') { self.text = self.text.replace('_', '') };

FLOAT_LIT: ((Dec? F_Dec F_Exp) | (Dec F_Dec) | (Dec F_Exp)) { self.text = self.text.replace('_',
'') };

fragment DEC: [1-9][0-9]*('_' [0-9]+)*;

fragment F_Dec: '.'[0-9]*;

fragment F_Exp: [eE][-+]?[0-9]+;
```

Dùng dữ kiện trên để trả lời cho các câu hỏi từ câu 2 đến câu 4

Câu 2: Trong các chuỗi sau đây, chuỗi nào được match với token INT_LIT?

A. 3_196_25

B. -20000

C. 09184

D. 3_017_298

Câu 3: Chuỗi 1_968.21e10 có hợp lệ hay không? Nếu hợp lệ, chuỗi token sinh ra là gì?

A. Hợp lệ; Dec F_Dec F_Exp

B. Không hợp lệ

C. Hợp lệ; FLOAT_LIT

D. Hợp lệ; INT_LIT F_Dec F_Exp

Câu 4: Trong đoạn code trên, từ khóa *fragment* có công dụng gì?

A. Không sinh ra những token dư thừa khi được nhúng vào các luật lexer

B. Là một luật hỗ trợ, có thể được nhúng vào các luật lexer khác, giúp code dễ đọc và dễ bảo trì hơn

C. Giúp hạn chế sự xung đột về luật khi có nhiều hơn 1 luật lexer match cùng 1 input

D. Tất cả đều sai

Dựa vào ảnh sau (mô tả các phương thức của class Token của ANTLR, sử dụng ngôn ngữ Java. Tên của các phương thức không thay đổi khi thay đổi ngôn ngữ đích), trả lời các câu hỏi từ câu 5 đến câu 6

Modifier and Type	Method	Description
int	getChannel()	Return the channel this token.
int	getCharPositionInLine()	The index of the first character of this token relative to the beginning of the line at which it occurs, 0..n-1
CharStream	getInputStream()	Gets the CharStream from which this token was derived.
int	getLine()	The line number on which the 1st character of this token was matched, line=1..n
int	getStartIndex()	The starting character index of the token This method is optional; return -1 if not implemented.
int	getStopIndex()	The last character index of the token.
String	getText()	Get the text of the token.
int	getTokenIndex()	An index from 0..n-1 of the token object in the input stream.
TokenSource	getTokenSource()	Gets the TokenSource which created this token.
int	getType()	Get the token type of the token

Câu 5: Đoạn code ANTLR sau đây biểu diễn luật đặt tên biến trong ngôn ngữ lập trình C++:

ID: [a-zA-Z_][a-zA-Z0-9_]*;

Với ngôn ngữ đích là C++, câu lệnh nào được sử dụng để lấy chuỗi văn bản sau khi token ID được match?

A. this→getType()

B. this→getText()

C. this→getLine()

D. this→getChannel()

Câu 6: Giả sử có token ID mang số thứ tự 11 trong file .token. Câu lệnh nào trong Python dùng để kiểm tra xem một token nào đó có phải là ID hay không?

A. token.getType() == ID

B. token.getText() == 11

C. token.getChannel == ID

D. token.getType() == 11

Dựa vào ảnh sau (mô tả các thuộc tính của class CommonToken – class cha của class Token) để trả lời câu hỏi từ câu 7 đến câu 9

Fields

Modifier and Type	Field	Description
protected int	channel	This is the backing field for getChannel() and setChannel(int).
protected int	charPositionInLine	This is the backing field for getCharPositionInLine() and setCharPositionInLine(int).
protected static Pair<TokenSource, CharStream>	EMPTY_SOURCE	An empty Pair which is used as the default value of source for tokens that do not have a source.
protected int	index	This is the backing field for getTokenIndex() and setTokenIndex(int).
protected int	line	This is the backing field for getLine() and setLine(int).
protected Pair<TokenSource, CharStream>	source	This is the backing field for getTokenSource() and getInputStream().
protected int	start	This is the backing field for getStartIndex() and setStartIndex(int).
protected int	stop	This is the backing field for getStopIndex() and setStopIndex(int).
protected String	text	This is the backing field for getText() when the token text is explicitly set in the constructor or via setText(java.lang.String).
protected int	type	This is the backing field for getType() and setType(int).

Biết rằng trong Python, các field đều được giữ nguyên tên.

Câu 7: Trong ANTLR, token được phân loại dựa trên channel. Mặc định, tất cả token đều được đưa vào 1 channel, ký hiệu là DEFAULT_CHANNEL. Token thuộc channel này sẽ được đưa vào parser để xử lý.

Ngoài ra còn 1 channel nữa là HIDDEN_CHANNEL, parser sẽ bỏ qua các token thuộc channel này.

DEFAULT_CHANNEL và HIDDEN_CHANNEL là các static attribute của class Token. Trong ANTLR4, DEFAULT_TOKEN_CHANNEL và HIDDEN được sử dụng trong lệnh →channel để điều hướng token.

Chọn câu lệnh thích hợp để điền vào // TODO giúp token sau đây được đưa vào channel HIDDEN_CHANNEL, với ngôn ngữ đích là Python:

CMT_LINE: ‘//’ ~[\r\n]~ // TODO;

a) → skip

b) →channel(HIDDEN)

c) { self.channel = Token.HIDDEN_CHANNEL }

d) Tất cả đều sai

Câu 8: Để thay đổi kiểu của token, ta sử dụng thuộc tính nào trong class CommonToken?

A. text

B. type

C. charPositionInLine

D. line

Câu 9: Đoạn code sau dùng để biểu diễn string literal trong ngôn ngữ lập trình Elixir:

SINGLE_LINE_STRING : ‘"’ (ESCAPE | ‘\\"’ | ~[\\"])* ‘"’;

MULTI_LINE_STRING : ‘"""’ .*? ‘"""’;

SINGLE_LINE_CHARLIST : ‘\’ (ESCAPE | ‘\\\’ | ~[\\'])* ‘\’;

MULTI_LINE_CHARLIST : ‘\'\'\’ .*? ‘\'\'\’;

Không cần quan tâm về rule ESCAPE

Để loại bỏ các cặp dấu nháy (đơn với _CHARLIST, kép với _STRING), action block của các luật trên là gì?

A. { self.text = self.text[1:-1] } cho SINGLE_LINE, { self.text = self.text[3:-3] } cho MULTI_LINE

B. { self.text = self.text.replace(‘\’, ‘’) } cho _CHARLIST, { self.text = self.text.replace(‘"’, ‘’) } cho _STRING

C. { self.text = self.text.strip(‘\’) } cho _CHARLIST, { self.text = self.text.strip(‘"’) } cho _STRING

D. Tất cả đều sai

Dùng dữ kiện sau để trả lời các câu hỏi từ câu 10 đến 11

Trong ANTLR, lexer có nhiệm vụ chuyển đổi chuỗi ký tự đầu vào thành các token. Đối với một số ngôn ngữ lập trình, việc nhận dạng token phụ thuộc vào ngữ cảnh đang quét, chẳng hạn như khi xử lý chuỗi ký tự, comment, hoặc các vùng cú pháp đặc biệt. Để hỗ trợ việc thay đổi tập luật lexer theo từng ngữ cảnh, ANTLR cung cấp cơ chế **mode** cho lexer. Mode cho phép lexer chuyển đổi trạng thái trong quá trình phân tích từ vựng, từ đó áp dụng các luật token khác nhau tương ứng với từng ngữ cảnh. Mặc định, các token đều ở mode DEFAULT_MODE.

Câu 10: Xét đoạn code sau, dùng để hiện thực văn phạm cho block comment có thể lồng vào nhau:

BLOCK_CMT: '/*' → pushMode(COMMENT), skip;

mode COMMENT; // Bắt đầu mode COMMENT

COMMENT_START: '/*' → pushMode(COMMENT), skip;

COMMENT_END: '*/' → popMode, skip;

COMMENT_TEXT: . → skip;

Tác dụng của các hàm pushMode, popMode là gì?

A. Di chuyển giữa các mode với nhau

B. pushMode đẩy mode vào stack mode, popMode để lấy mode ở đỉnh stack ra ngoài

C. Xử lý các văn phạm lồng vào nhau theo kiểu đệ quy (recursive grammar)

D. Tất cả đều sai

Câu 11: ANTLR có một lệnh cho phép chuyển mode, đó là →mode(MODE). Vậy tại sao đoạn code ở trên lại không sử dụng →mode(COMMENT)?

A. →mode không hỗ trợ lưu lại mode cũ, do đó không thể quay trở lại mode cũ nếu gặp chuỗi '*/'

B. →mode chỉ hỗ trợ việc chuyển đổi từ ngữ cảnh này sang ngữ cảnh khác

C. →mode không thích hợp trong trường hợp recursive grammar

D. Tất cả đều sai

Câu 12: Khi nói về giai đoạn tokenize trong ANTLR4, có 1 số ý kiến như sau khi match chuỗi đầu vào:

i) ANTLR4 cố gắng match ngắn nhất có thể

ii) Nếu 2 luật lexer cùng match 1 input, luật nào đứng trước sẽ được match

iii) .*? là cú pháp non-greedy

iv) Có thể negate một fragment rule

v) Trong giai đoạn tokenize, left-recursion (ví dụ: ID: ID ‘a’ | ‘a’) sẽ sinh ra lỗi

Số phát biểu **SAI** là:

A. 1 B. 2 C. 3 D. 4

Đoạn code sau dùng để trả lời các câu hỏi từ câu 13 đến câu 14

```
INT_LIT: [0-9]*[0-9];                                                    // 1
FLOAT_LIT: [0-9]+ ‘.’ [0-9]+ ([Ee][+-]?[0-9]+)? | [0-9]+ ([Ee][+-]?[0-9]+); // 2
ERROR_CHAR: . { raise ErrorToken(self.text) };                           // 3
```

Câu 13: Giả sử chuỗi đầu vào là 1.e-3. Sau khi sinh ra các file cần thiết và thực thi chương trình, kết quả thu được là:

A. 1.e-3,<EOF>

B. Error Token: 1

C. Error Token: .e-3

D. Error Token: 1.e-3

Câu 14: Muốn kết quả thu được là *Error Token: 1* thì ta cần thay đổi chương trình ở dòng nào?

A. Dòng 1

B. Dòng 2

C. Không cần thay đổi

D. Dòng 3

Dữ kiện sau để trả lời các câu hỏi từ câu 15 đến câu 17

Trong C++, hằng số nguyên (integer literal) có thể có các dạng biểu diễn sau (tương ứng với các hệ đếm khác nhau):

- Hệ thập phân: bắt đầu bằng chữ số khác 0 (từ 1 đến 9), theo sau đó là 0 hoặc nhiều hơn các chữ số từ 0 đến 9

- Hệ bát phân: bắt đầu bằng chữ số 0, theo sau đó là 0 hoặc nhiều hơn các chữ số từ 0 đến 7

(Dịch 1 phần từ trang en.cppreference.com)

Câu 15: Giả sử luật lexer cho số nguyên ở hệ thập phân trong C++ là DEC_INTLIT. Biểu thức chính quy cho DEC_INTLIT là gì?

A. [0-9]*[0-9]

B. [0-9]+

C. [1-9][0-9]*

D. [1-9]+

Câu 16: Giả sử luật lexer cho số nguyên ở hệ bát phân trong C++ là OCT_INTLIT. Biểu thức chính quy cho OCT_INTLIT là gì?

A. ‘0’ | [0][0-7]+

B. [0-7]+

C. ‘0’ | [0][0-7]*

D. [0][0-7]*

Câu 17: Đoạn code nào trong ANTLR mô tả đúng địa chỉ IPv6 dạng đầy đủ?

Recall: IPv6 dạng đầy đủ gồm 8 nhóm, mỗi nhóm gồm 4 chữ số ở hệ thập lục phân (0-9, A-F hoặc a-f), ngăn cách nhau bởi dấu ‘:’. Với các nhóm có chữ số 0 đứng đầu, ta có thể lược bỏ đi (trừ trường hợp nhóm đó biểu diễn số 0)

A. IPv6: HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX;

fragment HEX: HEX_DIGIT HEX_DIGIT? HEX_DIGIT? HEX_DIGIT?;

HEX_DIGIT: [0-9A-Fa-f];

B. IPv6: HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX;

fragment HEX: HEX_DIGIT HEX_DIGIT? HEX_DIGIT? HEX_DIGIT?;

fragment HEX_DIGIT: [0-9A-Fa-f];

C. IPv6: HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX ‘:’ HEX;

fragment HEX: HEX_DIGIT+;

fragment HEX_DIGIT: [0-9A-Fa-f];

D. Tất cả đều sai

Câu 18. Cho chuỗi đầu vào: 2+3-10/4. Đoạn code ANTLR4 của luật lexer INTLIT nào sau đây thỏa mãn chuỗi token sinh ra là INTLIT,INTLIT,INTLIT,DIV_OP,INTLIT

A. INTLIT: [-+]?[0-9]*[0-9];

B. INTLIT: [-+][0-9]+;

C. INTLIT: [-+]?('0' | [0-9][0-9]+);

D. INTLIT: [-+]?[0-9]*;

Đoạn code sau đây để trả lời các câu hỏi từ câu 19 đến câu 20

```
lexer grammar SimpleLexer;

ID      : [a-zA-Z_] [a-zA-Z_0-9]* ;

INT     : [0-9]+ ;

WS      : [ \t\r\n]+ -> skip ;

COMMENT : '//' ~[\r\n]* -> skip ;
```

Giả sử chuỗi đầu vào là

count1 // This is a comment

42abc

Câu 19: Những phát biểu nào sau đây về hành vi của lexer trong ví dụ trên là **đúng**?

A. Lexer dừng việc phân tích khi gặp luật COMMENT.

B. Lexer luôn chọn luật khớp được chuỗi dài nhất có thể tại vị trí hiện tại.

C. Chuỗi 42abc được tách thành hai token vì không có luật nào khớp được toàn bộ chuỗi này.

D. Nếu luật INT được định nghĩa trước luật ID, thì 42abc sẽ được khớp thành một token duy nhất.

Câu 20: Những token nào sau đây sẽ được lexer sinh ra (**bỏ qua các token bị skip**)?

A. ID("count1"), INT("42"), ID("abc")
B. ID("count1"), INT("42abc")
C. ID("count1"), INT("42"), ID("abc"), COMMENT("// This is a comment")
D. ID("count1"), ID("42abc")